COMP 3059 – Capstone Project I

Software Requirements Analysis and Design Assignment

This assignment is an overview to gather the software needs with requirements analysis and help to proceed with the design.

The requirements analysis helps to break down functional and non-functional requirements to a basic design view to provide a clear system development process framework. It involves various entities, including business, stakeholders and technology requirements.

The design is the activity following requirements specification and before programming. Software design usually involves problem solving and planning a software solution.

To work on this assignment you could use the references and a sample template given below. The sample template can be customized to suit the nature of your project.

Reference Readings/Example:
http://www.uacg.bg/filebank/acadstaff/userfiles/publ_bg_397_SDP_activities_and_steps.pdf

www.cse.msu.edu/~chengb/RE-491/Papers/SRSExample-webapp.doc

Source for this template:
www.tricity.wsu.edu/~mckinnon/cpts322/cpts322-srs-v1.doc

1.0 Introduction

The Introduction section provides an overview of the system using software requirements analysis and design for the scope of the system.

EventMaster is a comprehensive event and ticket management platform designed to streamline the creation, scheduling, and sale of events. This document will define the services core capabilities, boundaries, and intended users.

1.1 Purpose

This document describes the high level software requirements for the system. It describes the what, not how, of the capabilities of the system for the intended audiences.

This document will outline the high-level software requirements for EventMaster. It will describe what our team plans to implement in regards to enabling event creation, ticket browsing, and administrative control for its intended users.

1.2 Scope

This explains what the proposed system will and will not do. Describe relevant benefits, objectives and goals. The description of scope should be consistent with the Project Plan.

In scope	Out of scope
Secure User Registration and Authentication Users will be able to create accounts, log in securely, and access personalized features	Native Mobile Application Deployment
Event Setup & Lifecycle Management Organizers will be able to create and edit events with configurable details	Integration with External Payment Gateways
Event and ticket Exploration, Seat Selection & Purchase Flow Users can explore events and their tickets, select seats, and complete a purchase flow	Multi-Language Interface Support

Administrative Dashboard for oversight and control Authorized users can manage events, monitor ticket activity, and oversee user roles through secure interfaces	Fraud Detection Algorithms
Functional Consumer Support The system will provide functional support features such as FAQs or contacts to assist users	QR Code Generation for Ticket Validation
	Connectivity with External Marketing Platforms

2.0 System Overview

The System Overview section introduces the system context and design.

2.1 Project Perspective

The Project Perspective describes the context and origin of the system by defining whether the system is:

- a follow-on member of a system family
- a replacement for existing systems, or
- a new self-contained system.

EventMaster is a replacement for existing event and ticket management services. It is designed to depend on external services such as authentication providers and potential third-party APIs. EventMaster presents itself as a more streamlined and transparent alternative to other commercial services like TicketMaster and Eventbrite.

2.2 System Context

The System Context describes the resulting software within the business case, including strategic issues in which the system is involved or which it specifically addresses.

EventMaster provides a secure, streamlined interface for browsing/creating events, purchasing tickets, and managing events/bookings.

EventMaster aims to solve problems such as:

Complex User Experience	Existing platforms bombard users with messy interfaces and confusing navigation. This affects every user, leading to lower user engagement

	and low event turnouts
Lack of Transparency	Hidden fees affect all ticket buyers and lead to low user trust and reduced sales
Limited Integration	Existing systems lack integration with payment options and analytic services
Security Concerns	Popular ticket services have been hacked, exposing vulnerabilities in ticket authentication and the protection of user data

2.3 General Constraints

General Constraints identify any business or system constraints that will impact the manner in which the software is to be:

- specified
- designed
- implemented, or
- tested.

- **Financial Limitations -** The initial development budget is constrained, which restricts the implementation of advanced features such as fraud detection, QR code generation, and external marketing integrations.

- **Time Restrictions** - The system must be developed within a single semester. As a result, prioritization of certain features will be necessary to ensure that core functionality is delivered within the timeframe.

- **Resource Constraints** - The team consists of 4 members. Due to limited personnel, certain advanced features may be excluded to maintain focus on essential deliverables.

- **Regulatory Compliance** - The service must comply with applicable data protection regulations, including the Canadian Consumer Privacy Protection Act (CPPA). This requirement may influence design decisions to ensure secure handling of user data.

- **Third-Party Dependencies** - Payment processing will rely on external services such as PayPal, Apple Pay, Stripe, etc. These third-party integrations introduce dependencies that may affect system design, testing, and completion timelines.

2.4 Assumptions and Dependencies

List any assumptions that have been made during the initiation of the project. In addition, list any dependencies that may impact its success or the desired result.

Assumptions

- Users are familiar with basic web navigation and online purchasing workflows
- Event organizers have access to relevant event data and will utilize dashboard tools
- Administrators and organizers will constantly manage and monitor dashboards
- Users and organizers will access the platform through modern web browsers on Windows or macOS systems
- Digital payment methods (credit/debit cards, PayPal, Apple Pay, Stripe, etc) are accessible to users
- Simulated payment flows are acceptable for MVP functionality
- Stakeholders will provide feedback throughout the development lifecycle
- The platform will be hosted securely and comply with Canadian privacy regulations (e.g., CPPA)

Dependencies

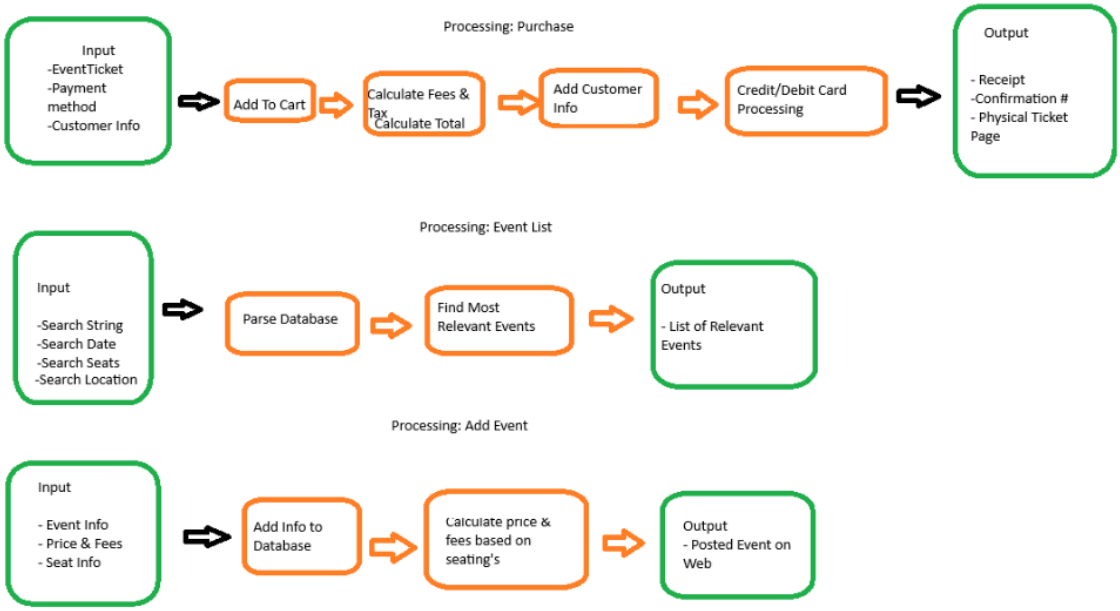
- Team availability and skill sets
- Completion of technical documentation and QA testing artifacts
- Web hosting infrastructure
- Third-party payment processing services
- Browser compatibility across major platforms
- Compliance with Canadian data protection laws

3.0 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.1 <Functional Requirement or Feature #1>

- Introduction: The ability to purchase, browse & add tickets.



...

3.2 Use Cases

Use Case #1 : Browse Concerts/ Events

Use Case #2: Navigate between pages

Use Case #3 : See available seats and their location

Use Case #4 : Bookmark events and get notifications

Use Case #5: Ability to share tickets

Use Case #6 : Ability to adjust font size

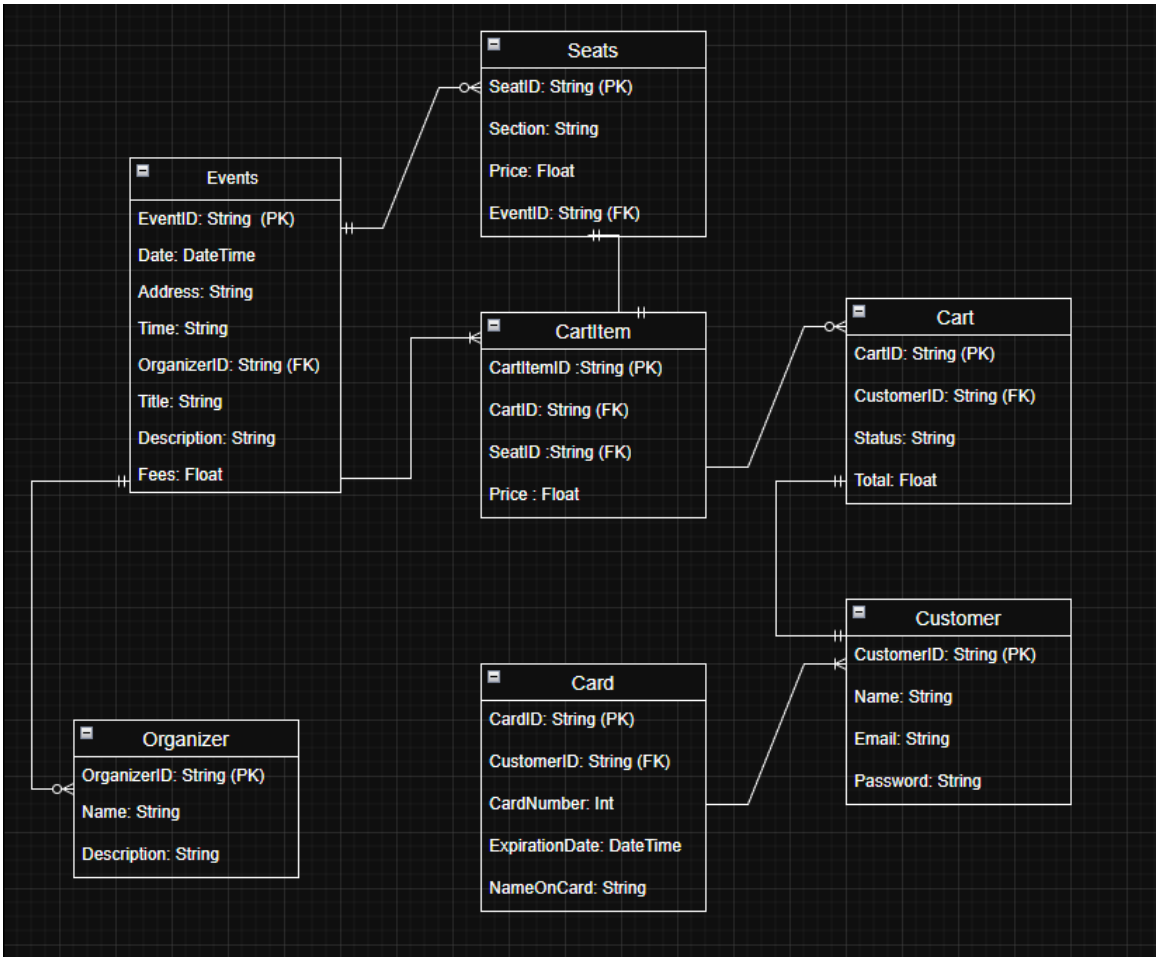
Use Case #7 : Create an event

Use Case #8: Monitor Ticket sales

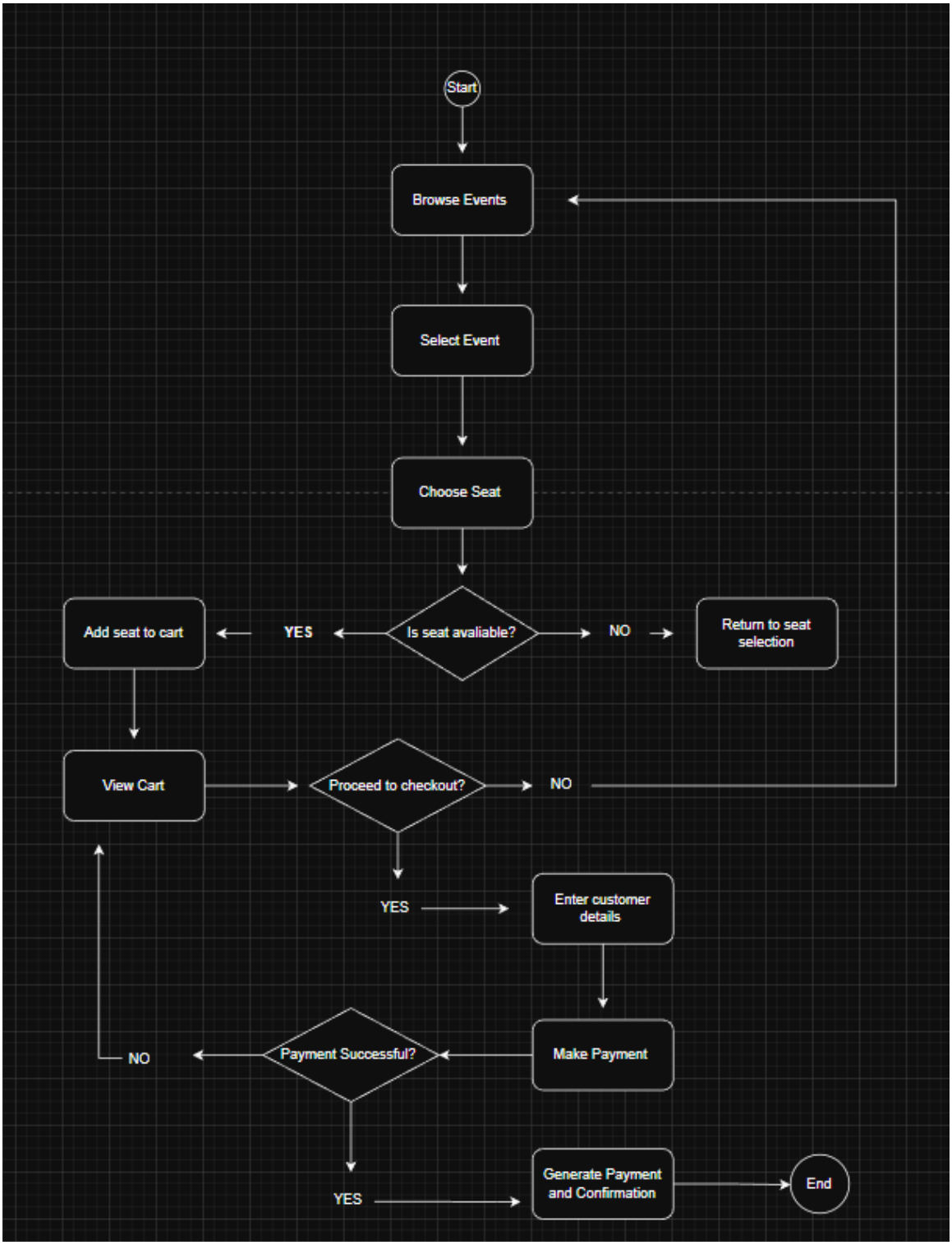
Use Case #9 : Adding Promo Codes

3.3 Data Modelling and Analysis

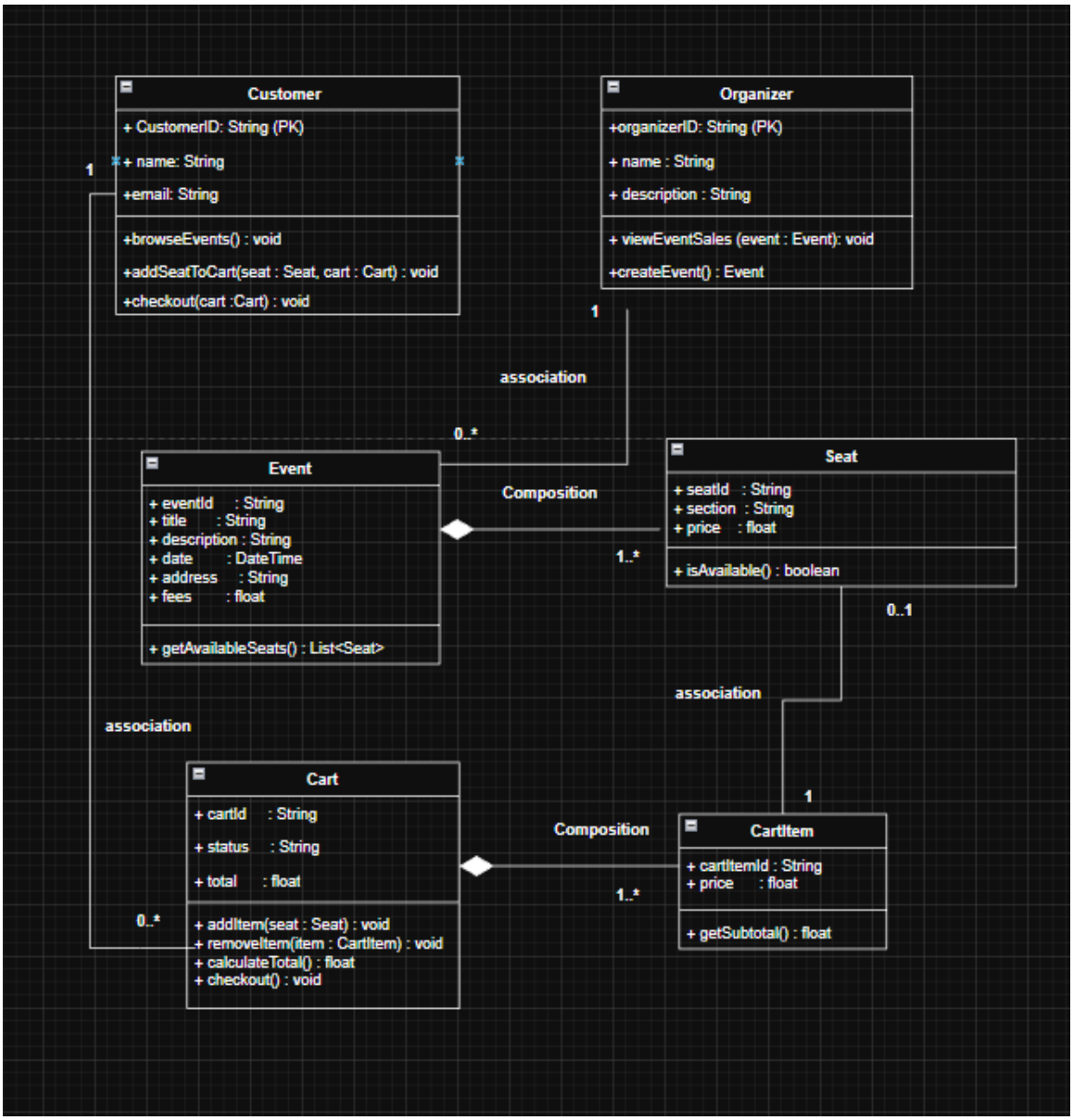
- Normalized Data Model Diagram



- Activity Diagrams

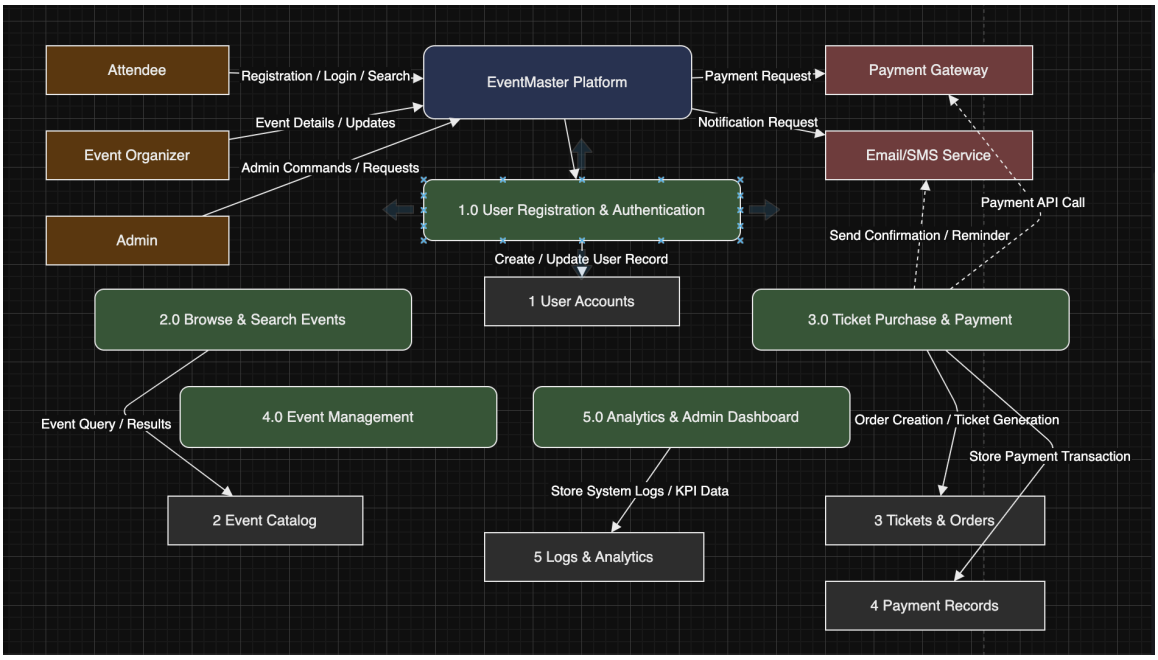


- Sequence Diagrams
- UML Class Diagram



3.4 Process Modelling

- Data Flow Diagram



4.0 Non-Functional Requirements

The non-functional requirements for a system are typically constraints on the functional requirements – that is, not what the system does, but how it does it (e.g. how quickly, how efficiently, how easily from the user's perspective, etc.).

Non-functional requirements may exist for any of the following attributes – Performance, Reliability, Availability, Security, Maintainability, Portability.

Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transactions shall be processed in less than a second, system downtime may not exceed 1 minute per day, etc).

Performance

- 95% of API requests should be processed in under 500ms
- Page load time shall not exceed 2 seconds on standard broadband
- Seat selection and ticket purchase flows shall complete in under 5 seconds

Reliability

- System shall maintain 99.9% uptime during operational hours
- All critical operations (e.g., ticket purchase, payment processing) shall be transactionally safe

Availability

- The system shall be accessible 24/7, with scheduled maintenance

Security

- Role-based access control shall restrict admin and organizer privileges
- Sensitive data (payment tokens, user credentials, etc.) should be stored using encryption

Maintainability

- Documentation shall be maintained for all public APIs and internal modules

Portability

- Frontend should support popular browsers such as Chrome, Firefox, Safari, and Edge on desktop and mobile
- Database should be compatible with PostgreSQL and MySQL

5.0 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc?

EventMaster will use a relational database such as PostgreSQL or MySQL to manage structured data for events, users, tickets, etc.

Data formats: UTF-8 encoding for text fields, ISO 8601 for date/time values, Boolean flags for status indicators, JSON fields for flexible metadata (event tags, etc.)

Storage capabilities: Must be able to support many events, tickets, and users

Data retention: Event and transaction data retained for a number of years while user data retained until deletion of account

Data integrity: Enforced via foreign key constraints, unique indexes, and transactional consistency

Security: Role-based access control, encrypted fields for sensitive data (various types of tokens)

6.0 Other Requirements

Additional requirements, if any.

Scalability: Ensuring the platform is able to support thousands of concurrent users. For future compatibility, potentially creating an auto-scaling system when CPU workloads are overloaded past a certain usage % for more than X amount of time.

7.0 Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Backend Developer	Jaeden Salandanan	Jaeden Salandanan	Nov 14, 2025
Frontend Developer	Regina Slonimsky	Regina Slonimsky	Nov 14, 2025
QA Specialist/Support	Vincent Luu	Vincent Luu	Nov 14, 2025